

Assignment 2A: n8n Template Analysis

Calculate LLM Token Usage from AI Agents and log it to Google Sheets

Prepared by: Gayathri

Template: Calculate LLM Token Usage (author: Solomon, from the n8n template library)

1. Overview

This workflow measures how many tokens an AI Agent used in a run, and writes the result to a Google Sheet that estimates the cost. It solves a real problem: with AI agents, each run can trigger several LLM calls (including calls caused by tool use), and the agent's own output does not give a simple total. Without tracking, teams cannot see what each workflow costs.

The workflow gets around this by reading the execution metadata of the finished run through the n8n API, extracting token counts from every language-model call, summing them by model, and appending one row per model to a spreadsheet.

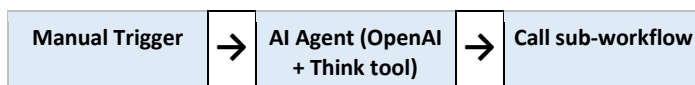
2. The Real-World Problem

- **Cost visibility:** LLM usage is billed per token, so teams running many agent workflows need to know the cost per workflow and per execution.
- **Hidden calls:** an agent that uses tools makes multiple model calls in one run. The final output does not show the total tokens.
- **Multiple providers:** the same pattern has to work for OpenAI, Gemini and Anthropic models.
- **Reporting:** finance or product owners want one table they can filter and sum, not raw execution JSON.

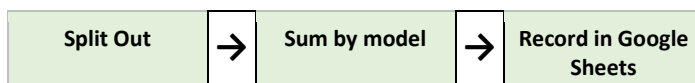
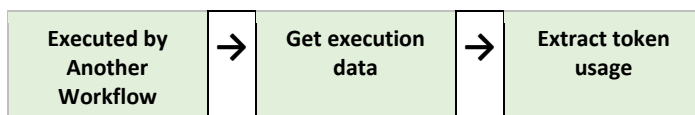
3. Workflow Flow

The canvas contains two connected parts. Part A is the example agent that does the work. Part B is the reusable sub-workflow that calculates and records token usage.

Part A: Example agent



Part B: Token usage sub-workflow



Part A hands over only the execution ID. Part B uses that ID to fetch the full record of the run, so the main workflow stays simple and the logic can be reused by any other agent workflow.

4. Node-by-Node Explanation

Node	Type	What it does	Why it is used / key settings
When clicking 'Test workflow'	Manual Trigger	Starts the workflow when you press Test.	Simple starter for a demo. In real use it would be replaced by the trigger of your own agent workflow (chat, webhook, schedule).
AI Agent	AI Agent (LangChain)	Runs the prompt: call the think tool twice, then output "This is a test workflow".	The prompt forces multiple model calls so there is enough token data to measure. Prompt type is "define" (fixed text).
OpenAI	Chat Model (gpt-4o-mini)	The language model the agent uses. This is the one connected to the agent.	Model node supplies the LLM. Cheap model chosen for testing.
Gemini and Anthropic	Chat Models (alternatives)	Alternative models, present but not connected.	Shows the pattern works across providers. Connect one instead of OpenAI to test it.
Think	Think tool	Gives the agent a place to reason before answering.	Creates extra model calls, which is what makes the token tracking meaningful.
Call sub-workflow	Execute Workflow	Runs Part B and passes the current execution ID.	Mode "each"; "Wait for sub-workflow" is off so the main run finishes first. Workflow ID is {{ \$workflow.id }} (this same workflow). Input execution_id = {{ \$execution.id }}.
When Executed by Another Workflow	Execute Workflow Trigger	Entry point of Part B; receives execution_id.	Defines the input field execution_id. Pinned test data (594878) is stored here for testing.
Get execution data	n8n (API)	Fetches the full execution record by ID.	Operation: Execution > Get. Needs an n8n API credential. Returns the run data of every node, including model calls.
Extract token usage data	Edit Fields (Set)	Builds two fields: execution_id and tokenUsage (an array).	tokenUsage is built with a JMESPath expression that finds every node run that used a language model and picks the model name and token counts (details in section 5).
Split Out	Split Out	Turns the tokenUsage array into one item per model call.	Needed so each call can be summed in the next step.

Node	Type	What it does	Why it is used / key settings
Sum Token Totals - aggregate by model	Summarize	Sums prompt tokens and completion tokens.	Grouped by workflow id, workflow name, model and execution_id, so each model gets one row per execution.
Record token usage	Google Sheets	Appends a row to the Executions sheet.	Writes timestamp, workflow id and name, execution id, model, input tokens and completion tokens. Cost columns are left to spreadsheet formulas.
Sticky Notes	Sticky Note	Documentation on the canvas.	Explain setup, pricing links, limitations and author info. They do not execute.

5. Key Expressions Explained

5.1 Passing the execution ID

`{{ $execution.id }}` returns the ID of the run that is currently executing. The sub-workflow receives it as `execution_id` and uses it to look up that same run. `{{ $workflow.id }}` returns the ID of the current workflow; it is used to call a sub-workflow that lives in the same workflow file.

5.2 Extracting token usage with JMESPath

The `$jmespath(...)` expression searches the execution JSON. In plain words it does this:

1. Look through the run data of all nodes (`data.resultData.runData`) and flatten the runs into one list.
2. Keep only the runs that contain a language-model output (`data.ai_languageModel`).
3. For each one, return the model name and the token usage.

It reads the model name from the response metadata, or from the input options as a fallback. It reads `tokenUsage`, or the `tokenUsageEstimate` when the provider does not return exact counts. The fallbacks are why one expression works for OpenAI, Gemini and Anthropic.

5.3 Timestamp

`{{ $now.format('yyyy-MM-dd HH:mm:ss') }}` stores when the row was recorded, in a consistent sortable format.

6. Design Decisions Worth Noting

- **Call the sub-workflow last and do not wait for it.** If the main run is still executing, the execution data is incomplete and large. Placing the call at the end with waiting turned off gives cleaner data and an easier token count.
- **Separate sub-workflow.** The tracking logic is reusable. Any agent workflow only needs one Execute Workflow node at its end.

- **Costs calculated in the spreadsheet, not in n8n.** The sheet has an LLM Pricing tab you update as prices change, so the workflow does not need editing.
- **Provider-agnostic.** The fallbacks in the JMESPath expression handle different response formats.
- **Pinned data for testing.** The sub-workflow trigger has pinned test data (execution ID 594878). Unpin it for real runs, otherwise the sub-workflow keeps reading that old execution instead of the ID sent by the main workflow.

7. How to Rebuild It

Prerequisites

- An n8n API key from your instance (Settings > n8n API), saved as an n8n API credential.
- Google Sheets credentials, and a copy of the template spreadsheet with an Executions tab and an LLM Pricing tab.
- An API key for at least one LLM provider.

Build order

1. Create a new workflow. Add a Manual Trigger and an AI Agent. Set the prompt type to "Define" and enter a prompt that forces two Think calls.
2. Attach a chat model (for example OpenAI gpt-4o-mini) and the Think tool to the agent.
3. Add an Execute Workflow Trigger node on the canvas (this starts Part B). Add an input field named `execution_id`.
4. Add the n8n node: resource Execution, operation Get, execution ID = `{{ $json.execution_id }}`. Choose your n8n API credential.
5. Add an Edit Fields node. Create `execution_id` and `tokenUsage`. For `tokenUsage`, use the JMESPath expression from section 5.2. Keep the workflow id and name fields.
6. Add Split Out on the `tokenUsage` field, keeping all other fields.
7. Add Summarize: sum prompt tokens and completion tokens, split by workflow id, name, model and `execution_id`.
8. Add Google Sheets with operation Append to the Executions sheet and map each column from the Summarize output.
9. Go back to Part A. Add an Execute Workflow node after the agent: mode "each", workflow ID = `{{ $workflow.id }}`, input `execution_id` = `{{ $execution.id }}`, and turn off Wait for Sub-Workflow Completion.
10. Test with a real run (no pinned data) and confirm a new row appears in the sheet.

8. Test Checklist

Fill in the Actual result column after you run each test.

#	Scenario	Expected result	Actual result	Status
1	Run the example agent with OpenAI connected	A row is added with model gpt-4o-mini and non-zero input and completion tokens		
2	Check the Think tool calls	Token totals include the extra calls caused by the two Think tool uses		
3	Swap OpenAI for Gemini, then Anthropic	Rows are added with the correct model names; counts are present		
4	Run with pinned data still on the sub-workflow trigger	Old execution (594878) is read instead of the new one. Confirms why pinning must be removed		
5	Remove or break the n8n API credential	Get execution data fails; no row is added; error is visible in executions		
6	Run twice in a row	Two separate rows with different execution IDs		
7	Check the sheet cost columns	Cost formulas fill from the LLM Pricing tab		

9. Limitations (from the template)

- Prompt caching is not accounted for, so the cost estimate can be higher than the real bill.
- Only tested with text and images, not audio or video.
- Costs in the sheet are estimates, not invoices. Batch requests and other discounts are not reflected.
- Other providers may need a change to the Extract token usage data node.

10. Possible Improvements

- Add an Error Trigger workflow that sends an alert when the token tracking fails.
- Add an IF node that sends a Slack or email alert when one execution uses more tokens than a set limit.
- Look up model prices inside n8n and write the cost directly, for cases where spreadsheet formulas are not wanted.
- Add a daily summary workflow that reads the sheet and reports total spend per workflow.

11. Conclusion

This template shows a practical pattern: instead of trying to get usage data from inside the agent, read the finished execution's metadata through the n8n API, process it with a small chain of data nodes, and store it where non-developers can analyse it. It uses a trigger, an agent, a sub-workflow, API access, data transformation and a spreadsheet integration, so it covers most of the core ideas of n8n in a small workflow.

Assignment 2B: n8n Template Analysis

Testing Multiple Local LLMs with LM Studio

Prepared by: Gayathri

Template: Testing Multiple Local LLM with LM Studio (n8n template library, tagged Training and Template)

1. Overview

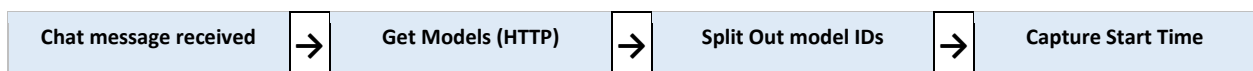
This workflow sends the same prompt to every model loaded in a local LM Studio server, one model after another. For each model it records the response, the time taken, and text-quality measures (word count, sentence count, average lengths and a readability score), then saves everything to a Google Sheet so the models can be compared side by side.

2. The Real-World Problem

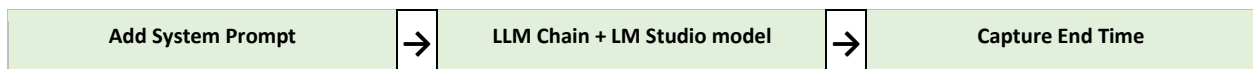
- **Choosing a model:** teams that run local LLMs (for privacy or to avoid per-token costs) need evidence for which model suits a task, not just a guess.
- **Fair comparison:** every model should get the same prompt and the same system instruction, so differences come from the model.
- **Measurable quality:** "easy to read" is subjective. A readability score and simple text statistics give numbers that can be compared.
- **Repeatability:** results go into a sheet, so you can run many prompts and review all runs together.

3. Workflow Flow

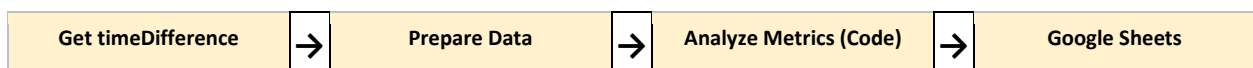
Stage 1: Get the list of models and prepare each run



Stage 2: Run the prompt on each model



Stage 3: Measure and save



The key idea is the Split Out node. LM Studio returns one list of models. Splitting it into one item per model makes every following node run once per model, so one chat message produces one result row per model.

4. Node-by-Node Explanation

Node	Type	What it does	Why it is used / key settings
When chat message received	Chat Trigger	Opens a chat window and starts the workflow when you send a message.	The message text (chatInput) is the prompt to test. A chat trigger makes it quick to try many prompts.
Get Models	HTTP Request	Calls GET /v1/models on the LM Studio server to list the loaded models.	URL http://0.0.0.0:1234/v1/models, 10 second timeout. "On error: continue (error output)" is set, but the error output is not connected to anything.
Extract Model IDs to Run Separately	Split Out	Splits the "data" array into one item per model.	This turns one list into several runs, one per model.
Capture Start Time	Date & Time	Stores the current time in startDateTime.	Marks when the model call begins, for timing.
Add System Prompt	Edit Fields (Set)	Adds system_prompt: "Ensure that messages are concise and to the point readable by a 5th grader."	Gives every model the same instruction. "Include other fields" keeps the model ID flowing through.
LLM Response Analysis	Basic LLM Chain	Sends the chat message to the model with the system prompt and returns the answer as text.	Prompt type "define": text = chatInput, system message = system_prompt.
Run Model with Dynamic Inputs	OpenAI Chat Model (sub-node)	The language model for the chain, pointed at LM Studio.	LM Studio mimics the OpenAI API, so the OpenAI node works with Base URL http://0.0.0.0:1234/v1. Model = the current item's ID. Temperature 1, Top P 1, presence penalty 0, timeout 250000 ms.
Capture End Time	Date & Time	Stores the current time in endDateTime.	Marks when the answer came back.
Get timeDifference	Date & Time	Calculates the time between start and end.	Operation: Get Time Between Dates, start from Capture Start Time and end from endDateTime.
Prepare Data for Analysis	Edit Fields (Set)	Collects response, prompt, model, start time, end time and time difference into one item.	Pulls values from earlier nodes with expressions so the next steps get a clean record.
Analyze LLM Response Metrics	Code (JavaScript)	Calculates word count, sentence count, average sentence length, average word length and a readability score.	Pure text analysis in code, no extra services needed (see section 5).
Save Results to Google Sheets	Google Sheets	Appends one row per model to Sheet1.	Operation: Append, with each column mapped to a field. Optional: you can delete it and read results by hand.
Sticky Notes	Sticky Note	Setup steps, tuning tips, readability ranges and a pro tip.	Documentation on the canvas only.

5. Key Expressions and Logic Explained

5.1 Choosing the model for each run

The model field of the chat model node uses the ID of the current item from the Split Out node. Because the workflow runs once per item, each pass uses a different model.

5.2 Reading values from earlier nodes

Expressions such as `$('#Capture Start Time').item.json.startDateTime` and `$('#When chat message received').item.json.chatInput` fetch a value from a specific earlier node for the matching item. This is how the Set node collects data that was created at different steps.

5.3 Text metrics in the Code node

- **Word count:** trim the text and split it on whitespace.
- **Sentence count:** match pieces of text ending in . ! or ?.
- **Average sentence length:** total words in those sentences divided by the number of sentences.
- **Average word length:** total characters divided by number of words.
- **Readability:** $206.835 \text{ minus } 1.015 \text{ times (words per sentence) minus } 84.6 \text{ times (syllables per word)}$. Syllables are estimated by counting groups of vowels. This is the Flesch Reading Ease formula, so a higher score means easier text.

5.4 How to read the readability score

Score	Meaning
90 to 100	Very easy to read (5th grade or below)
80 to 89	Easy to read (6th grade)
70 to 79	Fairly easy to read (7th grade)
60 to 69	Standard (8th to 9th grade)
50 to 59	Fairly difficult (10th to 12th grade)
30 to 49	Difficult (college)
0 to 29	Very difficult (college graduate)
Below 0	Extremely difficult (post-graduate)

5.5 Tuning settings

The chat model node exposes Temperature (randomness: higher is more varied, lower is more focused), Top P (limits the pool of likely words) and Presence Penalty (encourages new wording). Changing these lets you test how each setting changes the output of the same model.

6. Points to Check Before Relying on the Results

While reading the workflow JSON I noticed several things that are worth verifying in your own run. These are observations from reading the configuration, not results from running it.

- **Timing may not be per model.** n8n normally runs each node for all items before moving to the next node. That means Capture Start Time runs for every model first, the LLM chain then runs all models, and only afterwards does Capture End Time run. If so, the recorded times would reflect the whole batch, not each model. Wrapping the middle steps in Loop Over Items with a batch size of 1 would time each model separately.
- **Time difference is read in days.** Prepare Data for Analysis stores timeDifference.days, and the Get timeDifference node has no unit option set. A model answer takes seconds, so the value is likely to show as 0. Set the unit to seconds or milliseconds and read that field instead.
- **Two places hold the server address.** The Get Models node and the chat model node each contain the URL (both show 0.0.0.0:1234), while the sticky note mentions 192.168.1.1. Update both nodes, and use localhost, 127.0.0.1 or the real LAN IP of the LM Studio machine.
- **n8n must be able to reach LM Studio.** A local server is only reachable from n8n running on the same machine or network. A cloud-hosted n8n cannot reach a localhost address without a tunnel.
- **Sheet headers must match exactly.** The column mapping contains names with trailing spaces and spelling quirks ("Response ", "Time Recieved ", "Total Tlme spent ", "Tlme Sent"), and they differ from the headers listed in the sticky note. Create your sheet headers to match, or refresh and re-map the columns in the Sheets node.
- **Errors are not handled.** Get Models continues on error but its error output goes nowhere, so a stopped LM Studio server fails without a clear message.

7. How to Rebuild It

Prerequisites

- LM Studio installed, with at least two models downloaded and loaded, and its local server started (the template uses port 1234).
- An n8n instance that can reach that server.
- An OpenAI-type credential in n8n for the chat model node. LM Studio does not check the key, so a placeholder value is typically enough.
- A Google Sheet with headers matching the column mapping, and Google Sheets credentials.

Build order

1. Add a Chat Trigger node.
2. Add an HTTP Request node: GET to your server address followed by /v1/models, timeout 10000 ms. Run it once to see the model list.
3. Add a Split Out node on the field "data" so each model becomes its own item.
4. Add a Date & Time node (current date) with output field startDateTime.
5. Add an Edit Fields node that creates system_prompt, with "Include other fields" turned on.

6. Add a Basic LLM Chain. Set the text to the chatInput from the trigger and the system message to {{ \$json.system_prompt }}.
7. Attach an OpenAI Chat Model to the chain. Set Base URL to your server address plus /v1 and set Model by expression to the current item's id.
8. Add another Date & Time node with output field endDateTime, then a Date & Time node using Get Time Between Dates (start = startDateTime, end = endDateTime). Choose seconds as the unit.
9. Add an Edit Fields node that gathers llm_response, prompt, model, start_time, end_time and time_diff.
10. Add a Code node with the text analysis logic from section 5.3.
11. Add a Google Sheets node (Append) and map each field to its column.
12. Open the chat, send a test prompt, and check that the sheet gets one row per loaded model.

8. Test Checklist

Fill in the Actual result and Status columns after you run each test.

#	Scenario	Expected result	Actual result	Status
1	Run Get Models with LM Studio server running	Returns a data array with the IDs of all loaded models		
2	Load two or more models and send one prompt	One row per model is added to the sheet		
3	Load only one model	Exactly one row is added		
4	Check the Time Spent column	Shows a realistic time for each model, not 0 and not identical for all models		
5	Compare readability scores between models	Scores differ by model and fall in a sensible range (see section 5.4)		
6	Change the system prompt (for example, ask for technical language)	Readability scores drop compared with the 5th-grade prompt		
7	Stop the LM Studio server and send a prompt	Workflow stops with an error at Get Models; no row is added		
8	Rename a sheet header so it no longer matches	The column is empty or the node errors, confirming the mapping must match		

9. Limitations

- The readability score uses a rough syllable estimate based on vowel groups, so treat it as a comparison tool, not an exact grade level.
- Readability and length do not measure whether the answer is correct. A model can score well and still be wrong.

- Temperature 1 gives varied answers, so one run per prompt is a small sample. Repeat runs for fairer comparison.
- Everything depends on the LM Studio server being running and reachable.

10. Possible Improvements

- Use Loop Over Items (batch size 1) so timing is captured per model.
- Store the server address once in a Set node and reuse it in both nodes.
- Connect the error output of Get Models to an alert, such as an email or Slack message.
- Load a list of test prompts from a sheet and run them all in one execution.
- Add token counts or a correctness check against expected answers.

11. Conclusion

This template turns n8n into a small evaluation harness: the same input goes to several subjects, the outputs are measured the same way, and the results are stored for comparison. It shows how a Chat Trigger, an HTTP Request, Split Out, Date & Time nodes, a Basic LLM Chain, a Code node and Google Sheets can work together. It also shows why checking details such as time units and per-item execution matters before trusting the numbers.

Assignment 2C: n8n Template Analysis

AI Visual Regression Testing for Websites

Prepared by: Gayathri

Template: Visual regression testing with an AI vision model (n8n template library)

1. Overview

This workflow checks websites for unexpected visual changes. It keeps a "base" screenshot of each page, takes a fresh screenshot on a schedule, and sends both images to an AI vision model (Google Gemini) that lists what changed: text, numbers, images, colors or positions. Pages with changes are collected into one report and created as an issue in Linear.

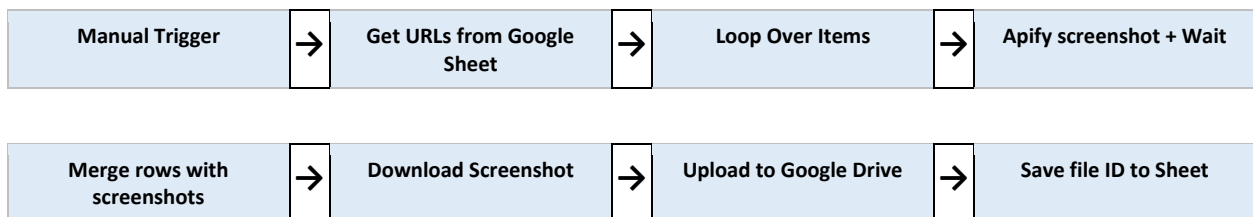
The workflow has two parts. Part A creates the base screenshots and is run manually only when the baseline needs updating. Part B runs on a weekly schedule and does the comparison.

2. The Real-World Problem

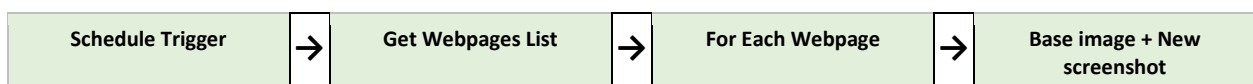
- **Visual bugs slip through.** Functional checks can pass while a page has a broken layout, a missing image, a wrong price or a changed color.
- **Manual checking does not scale.** Comparing screenshots of many pages by eye after every release is slow and easy to get wrong.
- **Pixel comparison is noisy.** Traditional tools flag tiny rendering differences. A vision model can be told what matters (content, layout, color) and what to ignore (text styling or casing).
- **Findings need to reach the team.** The result is posted as a ticket, so changes are tracked like any other bug.

3. Workflow Flow

Part A: Generate base images (manual run)



Part B: Run the visual regression test (weekly schedule)





The Google Sheet is the control table. It has the columns service, url and base. The base column holds the Google Drive file ID of each page's base screenshot, which links Part A to Part B.

4. Node-by-Node Explanation

4.1 Part A: Generate base images

Node	Type	What it does	Why it is used / key settings
When clicking 'Test workflow'	Manual Trigger	Starts Part A by hand.	Base images should only be refreshed on purpose, so there is no schedule.
Get URLs with Missing Base Images	Google Sheets	Reads the webpage list from Sheet1 of "Visual Regression List".	No filter is configured, so it reads every row, even though the name suggests only rows with a missing base image.
Loop Over Items	Split In Batches	Processes one URL at a time.	Keeps the screenshot service calls controlled. Its "loop" output goes to the screenshot step; its "done" output goes to the Merge node.
Run Webpage Screenshot	HTTP Request	POSTs the URL to an Apify screenshot actor and returns the result.	Apify "screenshot-url" actor, PNG, viewport width 1280, wait until DOM content loaded, Apify proxy on. Authenticated with a query-auth credential.
Wait	Wait	Pauses 2 seconds, then returns to the loop.	Avoids sending requests too fast to the external service.
Merge	Merge	Combines the original sheet rows with the screenshot results, matched by position.	Gives each item both its url and its screenshotUrl.
Download Screenshot	HTTP Request	Downloads the image from screenshotUrl.	Turns the link into a file the workflow can upload.
Upload to Drive	Google Drive	Saves the image in the base_images folder.	File name is the URL, URL-encoded, so each page has a recognizable file.
Update Base Image	Google Sheets	Writes the Drive file ID into the base column for that url.	Operation "Append or Update", matching on url. This links each page to its base image.

4.2 Part B: Run the visual regression test

Node	Type	What it does	Why it is used / key settings
Schedule Trigger	Schedule Trigger	Starts the test every week.	Set to weekly, Monday at 06:00, so checks run without anyone starting them.
Get Webpages List	Google Sheets	Reads the pages to test.	A filter on row_number = 2 is set, so it returns only one row. Remove it to test all pages.
For Each Webpage...	Split In Batches	Handles one page at a time.	The "loop" output starts the screenshot work; the "done" output starts the reporting.
Base Image	Google Drive	Downloads the base screenshot by file ID.	File ID comes from {{ \$json.base }}. Stored as binary property data_1.
Run Webpage Screenshot1	HTTP Request	Takes a fresh screenshot of the same URL through Apify.	Same settings as in Part A, so both images are captured the same way.
Download New Screenshot	HTTP Request	Downloads the new image as a file.	Response format "file", stored as binary property data_2.
Combine Screenshots	Merge	Brings the base and new image together in one item.	Combine by position, so one item holds data_1 (base) and data_2 (new).
Visual Regression Agent	Basic LLM Chain	Sends both images to the vision model and gets a list of differences.	System message tells the model what to look for and what to ignore. Two image messages read data_1 and data_2. Output parser is attached.
Google Gemini Chat Model	Chat Model (sub-node)	The vision model used by the chain.	Set to models/gemini-1.5-pro-latest. Any capable vision model can be used.
Structured Output Parser	Output Parser (sub-node)	Forces the answer into a JSON array.	Each item has type (text, number, image, color or position), description, previous_state and current_state.
Wait1	Wait	Pauses 1 second and loops back.	Light rate limiting between model calls.
Combine Row and Result	Edit Fields (Set)	Joins the sheet row with the model's output.	Raw JSON mode: copies the row from Get Webpages List and adds output.
Has Changes	Filter	Keeps only pages where the output array is not empty.	Condition: array length greater than 0 (strict type check).
Aggregate	Aggregate	Collects all remaining items into one list.	So one report is created for the whole run instead of one per page.
Create Report	Linear	Creates one issue listing all changes.	Title "Visual Regression Report" plus today's date. The description is built with an expression into markdown.

5. Key Expressions and Logic Explained

5.1 Loop outputs: "loop" versus "done"

A Split In Batches node has two outputs. The loop output sends one item at a time into the work steps, and the work steps connect back to the node. When every item has been processed, the done output fires once with all the results. Part A uses this to merge all screenshots at the end. Part B uses it to start the report.

5.2 Combining by position

The Merge nodes use "combine by position": item 1 is joined with item 1, item 2 with item 2, and so on. This works because both inputs keep the same order, but it would pair the wrong items if one side lost or reordered items.

5.3 Passing two images to the model

Each screenshot is stored as binary data under a different property name (data_1 for the base, data_2 for the new one). The LLM chain has two image messages, each pointing to one of those names, and the system message states that the first image is the base and the second is the test. That order is what lets the model say what changed.

5.4 Structured output

The parser schema makes the model return an array of objects with type, description, previous_state and current_state. An empty array is allowed and means no changes. Because the format is fixed, later nodes can filter on array length and build the report without parsing free text.

5.5 Building the report text

The Linear description expression loops over every page in the aggregated list. For each page it writes a heading with the URL, then one bullet per change in the form: description, expected previous_state, but got current_state. This produces a readable markdown report.

5.6 Merging row and result

`{ ...$('Get Webpages List').item.json, output: $json.output }` copies all columns of the original sheet row and adds the model output, so the report knows which URL each set of changes belongs to.

6. Points to Check Before Relying on the Results

These are observations from reading the workflow JSON, not results from running it.

- **Only one page is tested.** Get Webpages List filters on row_number = 2. This looks like a test setting; remove it to test every page.
- **Part A processes every row.** Despite its name, "Get URLs with Missing Base Images" has no filter, so a manual run regenerates all base images and uploads new files each time.
- **Model version.** The workflow uses gemini-1.5-pro-latest, an older model name. Choose a current vision-capable Gemini model in the chat model node if this one is unavailable.
- **Early screenshots can cause false alarms.** The screenshot is taken when the DOM content has loaded, with no delay. Images, fonts or lazy-loaded content may not have appeared yet, which can look like a change.

- **Dynamic content causes noise.** Ads, carousels, dates and cookie banners change between visits and may be reported as differences.
- **AI output varies.** The model may describe the same difference differently from run to run, or miss small details. Treat the report as a prompt for human review.
- **Service credentials needed.** Apify (screenshots), Google Drive and Sheets, Gemini, and Linear each need their own credentials. Linear can be swapped for Jira or Slack, as the template notes.

7. How to Rebuild It

Prerequisites

- An Apify account and API token, saved in n8n as a query-auth credential (Apify accepts the token as a query parameter named token).
- Google Drive and Google Sheets credentials, a Drive folder for base images, and a Google Sheet with the columns service, url and base.
- A Google Gemini API key (or another vision model).
- A Linear API key and team ID, or a Slack or Jira node instead.

Build Part A

1. Add a Manual Trigger and a Google Sheets node that reads all rows of your page list.
2. Add Loop Over Items. From its loop output, add an HTTP Request (POST to the Apify screenshot-url actor with the page URL in the body) followed by a Wait node that returns to the loop.
3. Add a Merge node (combine by position). Connect the Sheets node to input 1 and the loop's done output to input 2.
4. Add an HTTP Request to download the screenshot, a Google Drive node to upload it, and a Google Sheets node (Append or Update, match on url) to store the file ID in the base column.
5. Run Part A once and confirm each row now has a base value.

Build Part B

1. Add a Schedule Trigger (weekly) and a Google Sheets node to read the pages. Add a second manual trigger while testing.
2. Add Loop Over Items. From the loop output, start two branches: a Google Drive download of the base image (binary property data_1) and the Apify screenshot followed by a download (binary property data_2).
3. Merge both branches by position, then add a Basic LLM Chain. Attach a Gemini chat model and a Structured Output Parser with the change-list schema.
4. In the chain, write the system message (what to compare and ignore), then add two image messages that read data_1 and data_2.
5. Add a short Wait and connect back to the loop.
6. From the done output, add a Set node (row plus output), a Filter (output length greater than 0), an Aggregate node, and a Linear node that builds the markdown description.
7. Test with one page, then a page you deliberately changed, then all pages.

8. Test Checklist

Fill in the Actual result and Status columns after you run each test.

#	Scenario	Expected result	Actual result	Status
1	Run Part A with 2 or 3 URLs	A base image is uploaded for each URL and the base column in the sheet is filled		
2	Run Part B on an unchanged page	Model returns an empty array; no issue is created		
3	Change visible text on a test page and run Part B	Report lists a text change with previous and current values		
4	Change a color or move an element on the test page	Report lists a color or position change		
5	Replace an image on the test page	Report lists an image change		
6	Remove the row_number filter and use several pages	One report lists only the pages that changed, each under its own URL		
7	Test a page with dynamic content (carousel or date)	Note whether false positives appear		
8	Use an invalid URL or invalid Apify token	Workflow fails at the screenshot step; no report is created		
9	Change only text casing or font style	Not reported, because the system message says to ignore it		

9. Limitations

- A vision model gives a description, not a guaranteed pixel-exact comparison. It can miss small changes or report ones that are not real.
- Only the visible top of the page is captured with the current settings (no scroll to bottom), so changes lower on a long page are not tested.
- Results depend on third-party services (Apify, Gemini, Linear, Google) being available and within usage limits.
- Base images must be refreshed after intended design changes, otherwise every release is reported as a change.

10. Possible Improvements

- Add a screenshot delay or wait for network idle, and scroll to the bottom for full-page coverage.
- Filter the base-image list to rows where base is empty, so Part A only creates what is missing.
- Attach the base and new screenshot links to the Linear issue so reviewers can compare them quickly.

- Add error handling that alerts the team when a screenshot or model call fails.
- Use several viewport widths (desktop and mobile) to catch responsive layout problems.
- Point the workflow at a staging site and trigger it after each deployment instead of weekly.

11. Conclusion

This template automates a regression check that is normally done by eye. It shows many core n8n ideas in one workflow: scheduled and manual triggers, Google Sheets as a control table, loops with batch control, external API calls, binary file handling, Merge, an AI chain with image input and a structured output parser, Filter and Aggregate, and a ticketing integration. Its main lesson is that an AI step is most useful when the surrounding workflow gives it clean inputs, a strict output format and a human review step for the results.